

AIAC Assignment – 9.2

Name: K.Chaitanya

Ht no: 2303A51677,

Batch no:22

Task Description -1 (Documentation – Function Summary

Generation) Task:

Use AI to generate concise functional summaries for each Python function in a given script. Instructions:

- Provide a Python script to the AI.
- Ask the AI to write a short summary describing the purpose of each function.
- Ensure summaries are brief and technically accurate.
- Do not include code implementation details.

Prompt:

Generate concise functional summaries for each Python function.

Describe the purpose only.

Do not include implementation details for a number is even or odd

```
aiac-9.2.py > ...
1  '''Generate concise functional summaries for each Python function.
2  Describe the purpose only.
3  Do not include implementation details for the number which is even or odd'''
4  def is_even_or_odd(num: int) -> str:
5      """Determines if a given integer is even or odd."""
6      if num % 2 == 0:
7          return "Even"
8      else:
9          return "Odd"
10 n=int(input("Enter an integer: "))
11 print(f"The number {n} is {is_even_or_odd(n)}.")
12 '''Generate concise functional summaries for each Python function.
13 Describe the purpose only.Do not include implementation details for checking if a string is a palindrome.'''
14
```

OUTPUT:

```
/aiac-9.2.py"
PS C:\Users\chaitanya\New folder\AI assisted Coding> & C:/Users/chaitanya/AppDat
/aiac-9.2.py"
Enter an integer: 12
The number 12 is Even.
PS C:\Users\chaitanya\New folder\AI assisted Coding> |
```

Task Description -2 (Documentation – Logical Explanation for Conditions and Loops)

Task:

Use AI to document the logic behind conditional statements and loops in a Python program.

Instructions:

- Provide a Python program without comments.
- Instruct AI to explain only decision-making logic and loop behavior.
- Skip basic syntax explanations.

Prompt:

Explain the logical behavior of conditions and loops in this Python program. Focus only on decision-making and iteration flow. Do not explain syntax

```
'''Explain the logical behavior of conditions and loops in this Python program. Focus only on decision-making and iteration flow.
= Do not explain syntax eg: if, for, while, etc. and number is postive or negative.'''
def is_postive_or_negative(num: int) -> str:
    if num > 0:
        return "Positive"
    elif num < 0:
        return "Negative"
    else:
        return "Zero"
n = int(input("Enter an integer: "))
result = is_postive_or_negative(n)
print(f"The number {n} is {result}.")

# explanation:
# The function `is_postive_or_negative` evaluates the input number `num` through a series
# of conditions to determine if it is positive, negative, or zero. The decision-making process is structured as follows:
# 1. The first condition checks if `num` is greater than zero. If this condition is true, the function returns "Positive".
# 2. If the first condition is false, the function proceeds to the next condition,
#    which checks if `num` is less than zero. If this condition is true, it returns "Negative".
# 3. If neither of the above conditions is true (i.e., `num` is equal to zero), the function returns "Zero".
```

Task Description -3 (Documentation – File-Level Overview)

Task:

Use AI to generate a high-level overview describing the functionality of an entire Python file.

Instructions:

- Provide the complete Python file to AI.
- Ask AI to write a brief overview summarizing the file's purpose and functionality.
- Place the overview at the top of the

file. Expected Output -3:

A Python file with a clear and concise file-level overview at the beginning.

Prompt:

This Python program handles basic student information management.

It allows creation of student records, computes the average of marks, evaluates performance by assigning grades, and presents student details.

The system demonstrates structured programming using separate functions for each operation.

```
'''This Python program handles basic student information management.It allows creation of student records, computes the average of marks, evaluates performance by assigning grades, and presents student details.The system demonstrates structured programming using separate functions for each operation.'''

def create_student_record(name: str, marks: list) -> dict:
    return {"name": name, "marks": marks}
def calculate_average(marks: list) -> float:
    if not marks:
        return 0.0
    return sum(marks) / len(marks)
def assign_grade(average: float) -> str:
    if average >= 90:
        return "A"
    elif average >= 80:
        return "B"
    elif average >= 70:
        return "C"
    elif average >= 60:
        return "D"
    else:
        return "F"
def display_student_details(student: dict):
    name = student["name"]
    marks = student["marks"]
    average = calculate_average(marks)
    grade = assign_grade(average)
    print(f"Student Name: {name}")
    print(f"Marks: {marks}")
    print(f"Average: {average:.2f}")
    print(f"Grade: {grade}")

# Example usage:
student1 = create_student_record("Alice", [85, 92, 78])
display_student_details(student1)
```

Task Description -4 (Documentation – Refine Existing Documentation)

Task:

Use AI to improve clarity and consistency of existing documentation in Python code.

Instructions:

- Provide Python code containing basic or unclear comments.
- Ask AI to rewrite the documentation to improve clarity and consistency.
- Ensure technical meaning remains unchanged. Expected Output -4:

Python code with refined and improved documentation that is clear and consistent.

```
9 '''Write a Python program that implements the Binary Search algorithm to search for a target element in a sorted list.
10 The program should include a function for binary search, take input from the user, and display whether the element is found along with its position.
11 Use clear function structure and proper documentation'''
12 def binary_search(arr: list, target: int) -> int:
13     """Performs binary search on a sorted list to find the target element.
14
15     Args:
16         arr (list): A sorted list of integers.
17         target (int): The integer to search for.
18
19     Returns:
20         int: The index of the target element if found, otherwise -1.
21     """
22     left, right = 0, len(arr) - 1
23     while left <= right:
24         mid = left + (right - left) // 2
25         if arr[mid] == target:
26             return mid
27         elif arr[mid] < target:
28             left = mid + 1
29         else:
30             right = mid - 1
31     return -1
32
33 # Take input from the user
34 try:
35     sorted_list = list(map(int, input("Enter a sorted list of integers (space-separated): ").split()))
36     target_element = int(input("Enter the target element to search for: "))
37     result = binary_search(sorted_list, target_element)
38     if result != -1:
39         print(f"Element {target_element} found at index {result}.")
40     else:
41         print(f"Element {target_element} not found in the list.")
42 except ValueError:
43     print("Invalid input. Please enter integers only.")
44
```

OUTPUT:

```
PS C:\Users\chaitanya\New folder\AI assisted Coding> & C:/Users/chaitanya/AppData/Local/Programs/Python/Python311/Python.exe C:/Users/chaitanya/AppData/Local/Programs/Python/Python311/Python.exe /AIAC-8.5.py
Enter a sorted list of integers (space-separated): 1 2 3 56 89 90
Enter the target element to search for: 89
Element 89 found at index 4.
PS C:\Users\chaitanya\New folder\AI assisted Coding> 
```

Task Description -5 (Documentation – Prompt Detail Impact Study)

Task:

Study the impact of prompt detail on AI-generated documentation quality.

Instructions:

Create two prompts: one brief and one detailed.

- Use both prompts to document the same Python function.
- Compare the generated

outputs. Expected Output -5:

A comparison table highlighting differences in completeness, clarity, and accuracy of documentation.

```
'''Document the following Python function that checks whether a string is a palindrome.
Include the function purpose, input parameter description, return value, and behavior. Avoid explaining implementation details.'''
def is_palindrome(s: str) -> bool:
    """Checks whether the given string is a palindrome, ignoring spaces and case sensitivity.

    Args:
        s (str): The input string to be checked for palindrome properties.
    Returns:
        bool: True if the string is a palindrome, False otherwise.
    Behavior:
        The function normalizes the input string by removing spaces and converting it to lowercase before checking if it reads the same forwards and backwards.
    """
    cleaned_string = s.replace(" ", "").lower()
    return cleaned_string == cleaned_string[::-1]

s=input("Enter a string: ")
if is_palindrome(s):
    print(f'"{s}" is a palindrome.')
else:
    print(f'"{s}" is not a palindrome.')
```

